

firestore.rules

Visitor Management System — Firestore Security Rules — generated 2026-08-19

```
rules_version = '2';

service cloud.firestore {
  match /databases/{database}/documents {

    // ----- Helper functions -----
    // NOTE: these live INSIDE match /databases/{database}/documents so the
    // ${database} binding resolves correctly.

    function isSignedIn() {
      return request.auth != null;
    }

    function getRole(uid) {
      // Role is stored on the users/{uid} document.
      return get(/databases/${database}/documents/users/${uid}).data.role;
    }

    function isStaff() {
      return isSignedIn() && getRole(request.auth.uid) in
        ['super-admin', 'receptionist', 'security-officer'];
    }

    function isHost() {
      return isSignedIn() && getRole(request.auth.uid) == 'host';
    }

    function isSuperAdmin() {
      return isSignedIn() && getRole(request.auth.uid) == 'super-admin';
    }

    // Visit lifecycle: Pre-Registered -> Checked In -> Checked Out.
    // A visit may keep the same status or move forward only.
    function isValidStatusTransition(before, after) {
      return before == after
        || (before == 'Pre-Registered' && after == 'Checked In')
        || (before == 'Checked In' && after == 'Checked Out')
        || (before == 'Pre-Registered' && after == 'Checked Out');
    }

    // The only fields staff may change when checking a visitor in/out,
    // extending a visit, running the stay monitor, or recording identity and
    // facial verification results.
    function isVisitUpdateAllowed(before, after) {
      return isValidStatusTransition(before.status, after.status)
        && after.diff(before).affectedKeys().hasOnly([
          'status',
          'checkInTime',
          'expectedCheckoutTime',
          'duration',
          'durationMinutes',
          'photoUrl',
          'frequentVisitorId',
          'checkOutTime',
          'reminderSent',
          'reminderSentAt',
          'expiredEmailSent',
          'expiredEmailSentAt',
          'extendCount',
          'lastExtendedAt',
          'verificationStatus',
          'verificationMethod',
          'verifiedAt',
          'verifiedBy',
          'facialVerificationStatus',
          'facialVerificationAt',
        ]]);
    }
  }
}
```

```

}

// A Host may only set registrationEmailSent on their own Pre-Registered
// visit. They cannot touch any other field or progress the status.
function isHostRegistrationUpdateAllowed(before, after) {
    return before.registeredBy == request.auth.uid
        && before.status == 'Pre-Registered'
        && after.status == 'Pre-Registered'
        && after.diff(before).affectedKeys().hasOnly(['registrationEmailSent']);
}

// A Host may register a new visitor only as a Pre-Registered visit tied to
// their own account, with no facial/profile data attached (photoUrl).
function isHostRegistrationCreateAllowed(newData) {
    return newData.status == 'Pre-Registered'
        && newData.type == 'visitor'
        && newData.registeredBy == request.auth.uid
        && newData.keys().hasAll([
            'badgeNumber',
            'profileId',
            'expectedCheckoutTime',
            'registeredBy',
            'status',
            'type',
        ])
        && (!('photoUrl' in newData) || newData.photoUrl == null);
}

// A Host may create a new frequent-visitor profile only when it is tagged
// as host-created and carries no facial data (photos are captured by staff).
function isHostProfileCreateAllowed(newData) {
    return newData.createdBy == 'host'
        && (!('photoUrl' in newData) || newData.photoUrl == null);
}

// ----- Visitor visits (least-privilege) -----
// There is NO public/unauthenticated access to the visitors ledger.
// Badge numbers, personal information and facial capture data (photoUrl)
// are exposed only to authenticated staff (Reception / Security / Admin)
// or to a Host for the visits they themselves registered. Knowing a badge
// number grants nothing: the visitor database is never publicly readable.
// The visit lifecycle is Pre-Registered -> Checked In -> Checked Out.
match /visitors/{id} {
    // Staff record check-ins; a Host may pre-register a visitor under their
    // own account (Pre-Registered, no facial data).
    allow create: if isStaff()
        || (isHost() && isHostRegistrationCreateAllowed(request.resource.data));
    // Staff see every visit; a Host sees only the visits they registered.
    allow read: if isStaff()
        || (isHost() && request.data.registeredBy == request.auth.uid);
    // Staff may progress a visit (check-in, extend, reminders, check-out,
    // verification) but only within the allowed lifecycle and field set.
    // A Host may only mark their pre-registration email as sent.
    allow update: if (isStaff() && isVisitUpdateAllowed(resource.data, request.resource.data))
        || (isHost() && isHostRegistrationUpdateAllowed(resource.data, request.resource.data));
    // Audit integrity: visit records are never deleted by day-to-day staff.
    allow delete: if isSuperAdmin();
}

// ----- Frequent-visitor profiles -----
// Staff manage the profile directory (which contains facial/profile data).
// A Host may read profiles to reuse an existing one and may create a new
// host-owned profile (never with facial data). Unauthenticated visitors
// cannot read or write profiles.
match /frequentVisitors/{id} {
    allow read: if isStaff() || isHost();
    allow create: if isStaff()
        || (isHost() && isHostProfileCreateAllowed(request.resource.data));
    allow update, delete: if isStaff();
}

// ----- Staff-only directories -----
match /deliveries/{id} {
    allow read, create, update, delete: if isStaff();
}

```

```

match /hosts/{id} {
  // Staff manage the directory. A Host can read only their own linked
  // directory record (matched by userId == their auth uid).
  allow read: if isStaff() || (isHost() && resource.data.userId == request.auth.uid);
  allow create, update, delete: if isStaff();
}

match /contractors/{id} {
  allow read, write: if isStaff();
}

match /preRegistrations/{id} {
  allow read, write: if isStaff();
}

// ----- App settings (Super Admin writes; readable by all) -----
// Publicly readable so the check-in UI can show the organization name and
// default visit duration. Contains only non-sensitive configuration.
match /appSettings/{id} {
  allow read: if true;
  allow write: if isSuperAdmin();
}

// ----- Users / roles -----
// A user may read their own record; Super Admins manage everyone.
// The first Super Admin must be created in the Firebase Console
// (Authentication -> Add user, then Firestore -> users/<uid> with
// role "super-admin"). In-app signups can only create Receptionists,
// so no account can ever elevate its own role.
match /users/{uid} {
  allow read: if isSignedIn() && (isSuperAdmin() || uid == request.auth.uid);
  allow create: if isSignedIn()
    && uid == request.auth.uid
    && request.resource.data.role == 'receptionist';
  allow update, delete: if isSuperAdmin();
}
}
}

```